

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА — Российский технологический университет»
РТУ МИРЭА



ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
«Принцип работы таймера микроконтроллера. Расчёт времени таймера»
по дисциплине «Программирование киберфизических систем»

Выполнил студент группы ЭФБО-10-
23

Ефремов А.И.

Принял к.т.н., доцент
Сухатерин А.Б.

Москва 2026

СОДЕРЖАНИЕ

Введение	3
1 Теоретические сведения	4
2 Практическая реализация	7
Заключение	10
Список использованных источников	11

ВВЕДЕНИЕ

Аппаратный таймер — периферийный модуль микроконтроллера, который отсчитывает импульсы тактового сигнала независимо от выполнения прикладного кода. Таймеры применяются для формирования задержек, периодических событий, измерения длительности внешних импульсов, генерации ШИМ-сигналов и управления исполнительными устройствами [1].

Цель работы — изучить принцип работы аппаратного таймера STM32 и заменить системные задержки `HAL_Delay` в контроллере светофора собственной задержкой на базе TIM14.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Назначение таймеров STM32

Таймер содержит счётчик, который изменяется синхронно с тактовым сигналом. Процессор может читать текущее значение счётчика, сравнивать его с заданным значением или реагировать на переполнение. За счёт этого отсчёт времени не зависит от количества инструкций в основном цикле программы [1].

В STM32 используются несколько классов таймеров. SysTick входит в ядро Cortex-M и обычно используется HAL как системная временная база для HAL_Delay. Базовые таймеры предназначены для простого отсчёта времени и периодических событий. Таймеры общего назначения дополнительно могут работать с захватом входного сигнала, сравнением, ШИМ и однократными импульсами. Расширенные таймеры применяются, когда нужны комплементарные выходы и специальные режимы управления силовой нагрузкой. Отдельно используются таймеры низкого потребления и сторожевые таймеры [1, 2].

Для STM32C031C6 производитель указывает таймеры для управления ШИМ, четыре 16-разрядных таймера общего назначения, два сторожевых таймера и системный SysTick [3]. Конкретный набор возможностей определяется моделью микроконтроллера, поэтому при выборе таймера необходимо сверяться с её справочным руководством.

1.2 Предделитель, счётчик и переполнение

На вход таймера поступает тактовая частота F_{clk} . Предделитель PSC делит её на значение $PSC + 1$, поэтому частота счётчика вычисляется по формуле:

$$F_{cnt} = F_{clk} / (PSC + 1)$$

Счётчик увеличивается с частотой F_{cnt} и при достижении значения автоматической перезагрузки ARR начинает отсчёт заново. Для

периодического события период определяется выражением:

$$T = (ARR + 1) / F_{cnt}$$

При использовании таймера как свободно работающего счётчика достаточно запустить его и считывать текущее значение. Тогда длительность интервала определяется разностью текущего и сохранённого начального значения. Этот подход не требует обработчика прерывания, однако во время ожидания процессор остаётся в цикле проверки счётчика. В данной работе это допустимо, поскольку алгоритм светофора изначально также использовал блокирующие задержки.

1.3 Типы таймеров и выбор TIM14

Номера TIMx не образуют одну универсальную последовательность для всех STM32: набор модулей зависит от конкретного микроконтроллера. В STM32C031C6 имеются следующие аппаратные таймеры [3, 4]:

- TIM1 — расширенный таймер управления; он предназначен прежде всего для формирования ШИМ, в том числе для задач управления двигателем;
- TIM3 — четырёхканальный 16-разрядный таймер общего назначения;
- TIM14, TIM16 и TIM17 — одноканальные 16-разрядные таймеры общего назначения;
- SysTick — системный таймер ядра Cortex-M0+, используемый HAL для функции HAL_Delay;
- IWDG и WWDG — сторожевые таймеры, предназначенные для перезапуска программы при её зависании, а не для обычных задержек.

Таким образом, TIM1 образует отдельную группу расширенных таймеров, а TIM3, TIM14, TIM16 и TIM17 — группу таймеров общего назначения. Для светофора требовался только независимый миллисекундный счётчик: не нужны комплементарные ШИМ-выходы TIM1 и не нужно занимать SysTick, который обслуживает HAL_Delay. Поэтому можно было

выбрать любой свободный таймер из группы общего назначения, если он не задействован другой частью проекта.

Выбран TIM14. Он не используется схемой GPIO светофора, имеет программируемый 16-разрядный предделитель, 16-разрядный счётчик с автоматической перезагрузкой и один канал захвата, сравнения или ШИМ [4]. В данной работе используется только счётчик, поэтому возможностей TIM14 достаточно, а его применение создаёт отдельную от SysTick аппаратную временную базу.

2 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ

2.1 Расчёт параметров TIM14

В проекте системная частота HSI составляет 48 МГц. Для получения частоты счётчика 1 кГц выбран предделитель 48000 - 1. Расчёт имеет вид:

$$F_{cnt} = 48\,000\,000 / (47\,999 + 1) = 1\,000 \text{ Гц}$$

Следовательно, счётчик увеличивается на единицу каждую миллисекунду. В качестве периода установлен 0xFFFF, то есть максимальное значение 16-разрядного счётчика. Переполнение происходит через 65 536 отсчётов, или через 65,536 с. Наибольшая задержка в светофоре равна 10 000 мс, поэтому она значительно меньше периода переполнения.

Инициализация TIM14 и запуск счётчика приведены в Листинге 2.1. Функция HAL_TIM_Base_Start включает счётчик без использования прерываний.

Листинг 2.1 — Фрагмент программы на C

```
TIM_HandleTypeDef htim14;

static void MX_TIM14_Init(void) {
    __HAL_RCC_TIM14_CLK_ENABLE();

    htim14.Instance = TIM14;
    htim14.Init.Prescaler = 48000 - 1; // 48 МГц -> 1 кГц
    htim14.Init.CounterMode = TIM_COUNTERMODE_UP;
    htim14.Init.Period = 0xFFFF;
    htim14.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    htim14.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
    if (HAL_TIM_Base_Init(&htim14) != HAL_OK) {
        while (1) { }
    }
}

/* В функции main после MX_GPIO_Init(): */
MX_TIM14_Init();
HAL_TIM_Base_Start(&htim14);
```

2.2 Замена системной задержки

Вместо `HAL_Delay` в программе используется функция `TimerDelay`. В начале она сохраняет текущее значение регистра счётчика `TIM14`. Затем функция ожидает, пока беззнаковая разность текущего и начального значений не станет равна требуемой задержке.

Беззнаковое приведение к `uint16_t` важно при переполнении. Когда счётчик переходит от `0xFFFF` к нулю, арифметика 16-разрядного беззнакового значения сохраняет корректную разность. Поэтому задержки меньше 65 536 мс остаются правильными даже при переходе через переполнение. Реализация функции приведена в Листинге 2.2.

Листинг 2.2 — Фрагмент программы на C

```
static void TimerDelay(uint16_t ms) {
    uint16_t started_at = (uint16_t) __HAL_TIM_GET_COUNTER(&htim14);
    while ((uint16_t)(__HAL_TIM_GET_COUNTER(&htim14) - started_at) < ms) { }
}
```

Основная последовательность светофора и схема Wokwi не менялись: изменения касаются только временной базы. В исходном состоянии автомобилям разрешено движение, а пешеходам включён запрещающий сигнал. Это подтверждено Рисунком 2.1.

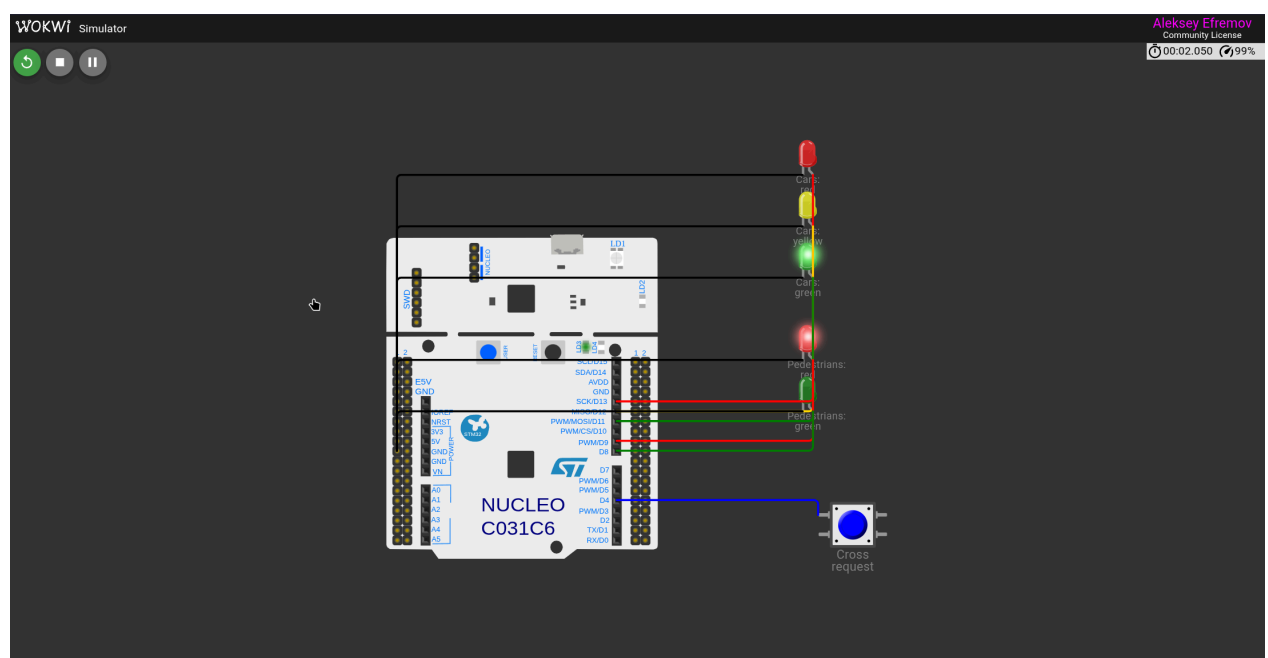


Рисунок 2.1 — Исходное состояние светофора с временной базой `TIM14`

После нажатия кнопки выполняется пешеходная фаза: автомобильный красный сигнал включён, пешеходный зелёный сигнал разрешает переход. Рисунок 2.2 показывает, что замена HAL_Delay на ожидание по TIM14 не изменила подключение устройств и ожидаемое состояние схемы.

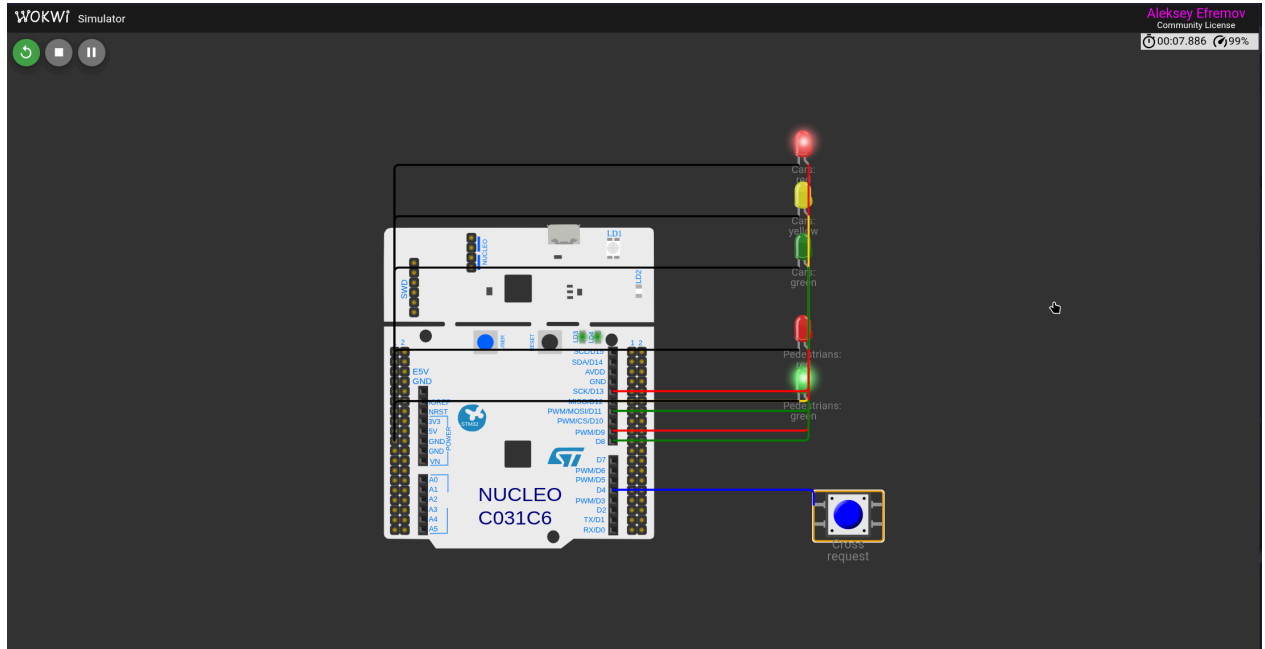


Рисунок 2.2 — Пешеходная фаза светофора с временной базой TIM14

2.3 Использование TimerDelay в алгоритме

В коде все интервалы из предыдущей реализации сохранены: 500 мс для мигания, 2 000 мс для жёлтого сигнала, 10 000 мс для пешеходной фазы и 10 мс для ожидания отпускания кнопки. Отличие состоит только в источнике времени. Пример замены задержки в цикле мигания приведён в Листинге 2.3.

Листинг 2.3 — Фрагмент программы на C

```
for (int i = 0; i < 6; i++) {  
    HAL_GPIO_TogglePin(GREEN_GPIO_Port, GREEN_Pin);  
    TimerDelay(500);  
}
```

ЗАКЛЮЧЕНИЕ

В ходе практической работы изучены назначение аппаратных таймеров STM32, принцип настройки предделителя и периода счётчика, а также способы использования таймера для формирования временных интервалов.

В контроллере светофора системные задержки `HAL_Delay` заменены собственной функцией `TimerDelay`, использующей свободно работающий счётчик TIM14. При системной частоте 48 МГц и предделителе 47 999 таймер формирует один отсчёт в миллисекунду. Полученная временная база не использует `SysTick` и сохраняет алгоритм работы светофора без изменения схемы GPIO.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Таймеры STM32: управление временем в микроконтроллере, примеры [Электронный ресурс]. — Skypro Wiki. — URL: <https://sky.pro/wiki/gamedev/ispolzovanie-tajmerov-na-stm32/> (дата обращения: 23.09.2026).
2. Программирование STM32 на C: освоение микроконтроллеров — путь к успеху [Электронный ресурс]. — Skypro Wiki. — URL: <https://sky.pro/wiki/gamedev/osnovy-programmirovaniya-stm32-na-yazyke-c/> (дата обращения: 23.09.2026).
3. STM32C031C6: product page [Электронный ресурс]. — STMicroelectronics. — URL: <https://www.st.com/en/microcontrollers-microprocessors/stm32c031c6.html> (дата обращения: 23.09.2026).
4. STM32C0 series advanced Arm-based 32-bit MCUs. RM0490 [Электронный ресурс]. — STMicroelectronics. — URL: https://www.st.com/resource/en/reference_manual/rm0490-stm32c0-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf (дата обращения: 23.09.2026).
5. Программирование STM32: от первого проекта до сложных систем [Электронный ресурс]. — Skypro Wiki. — URL: <https://sky.pro/wiki/gamedev/sozдание-prostyh-prilozhenij-na-stm32/> (дата обращения: 23.09.2026).
6. HAL TimeBase TIM Overview [Электронный ресурс]. — STMicroelectronics. — URL: https://dev.st.com/stm32cube-docs/stm32c5xx-hal-drivers/2.0.0/en/docs/drivers/hal_drivers/hal_common_drivers/hal_timebases/hal_timebase_tim/hal_timebase_tim_overview.html (дата обращения: 23.09.2026).
7. Программирование STM32: от основ к реальным проектам с примерами [Электронный ресурс]. — Skypro Wiki. — URL: <https://sky.pro/wiki/javascript/programmirovanie-stm32-vvedenie-i-primery/> (дата обращения: 23.09.2026).